
附件：

学号：	23009201247	
姓名：	郑墨涵	

《计算机安全导论》课程 自选实验报告

学 号： 23009201247

姓 名： 郑墨涵

班 级： 2303013

指导教师： 杨力、周路

日 期： 2025.12.23

计算机科学与技术学院

自选实验报告撰写要求

1. 不得抄袭、复制，不得滥用 AI 工具，一经发现取消实验成绩；
2. 正文小四字号，固定行距 20 磅，中文宋体，英文 Times New Roman；
3. 每个实验报告从新页开始；
4. 报告电子档命名规则：“《计算机安全导论》自选实验报告-学号-姓名.doc/.docx/.pdf”；
5. 实验报告作为实验验收的补充，仅提交实验报告未进行现场验收的没有实验部分成绩。

实验一：SHA1 算法 C 语言实现

一、实验原理

1.1 哈希算法概述

哈希算法是一种将任意长度输入数据转换为固定长度输出（哈希值）的单向函数。具有以下特性：

单向性：难以从哈希值反推原始数据

抗碰撞性：难以找到两个不同输入产生相同哈希值

雪崩效应：输入微小变化会导致输出巨大差异

1.2 SHA-1 算法原理

SHA-1（Secure Hash Algorithm 1）是由美国国家安全局设计，NIST 发布的密码哈希函数，输出为 160 位（20 字节）。

1.2.1 算法流程

1. **消息填充：**将原始消息填充至 512 位的倍数

先添加一个"1"位（0x80）

然后添加"0"位直到长度模 512 等于 448

最后添加 64 位原始消息长度

2. **初始化哈希值：**使用 5 个 32 位初始常量

$H_0 = 0x67452301$

$H_1 = 0xEFCDAB89$

$H_2 = 0x98BADCFE$

$H_3 = 0x10325476$

$H_4 = 0xC3D2E1F0$

3. **处理消息块：**每个 512 位块进行 80 轮运算

消息扩展：16 个 32 位字扩展为 80 个 32 位字

主循环：每轮使用不同的逻辑函数和常量

4. **输出结果：**将 5 个 32 位状态变量连接为 160 位哈希值

1.2.2 核心运算公式

每轮运算使用以下公式：

$temp = ROTL(a, 5) + f(b, c, d) + e + k + w[t]$

$e = d$

$d = c$

$c = ROTL(b, 30)$

$b = a$

$a = \text{temp}$

1.2.3 逻辑函数（分四阶段）

第 0-19 轮: $f = (b \& c) | ((\sim b) \& d)$ （选择函数）

第 20-39 轮: $f = b \wedge c \wedge d$ （奇偶函数）

第 40-59 轮: $f = (b \& c) | (b \& d) | (c \& d)$ （多数函数）

第 60-79 轮: $f = b \wedge c \wedge d$ （奇偶函数）

二、实验结果

2.1 实验环境

操作系统: Windows 11

开发工具: Visual Studio Code

编译器: GCC 13.2.0

测试数据: 字符串 "abc"

正在对 "abc" 进行 SHA1 加密...

>>> 进入 Transform (处理 64 字节块) <<<

Block Initial: A=67452301 B=efcdab89 C=98badcfe D=10325476 E=c3d2e1f0

[Round 00]	W=61626380	A=0116fc33	B=67452301	C=7bf36ae2	D=98badcfe	E=10325476
[Round 01]	W=00000000	A=8990536d	B=0116fc33	C=59d148c0	D=7bf36ae2	E=98badcfe
[Round 02]	W=00000000	A=a1390f08	B=8990536d	C=c045bf0c	D=59d148c0	E=7bf36ae2
[Round 03]	W=00000000	A=cdd8e11b	B=a1390f08	C=626414db	D=c045bf0c	E=59d148c0
[Round 04]	W=00000000	A=cfd499de	B=cdd8e11b	C=284e43c2	D=626414db	E=c045bf0c
[Round 05]	W=00000000	A=3fc7ca40	B=cfd499de	C=f3763846	D=284e43c2	E=626414db
[Round 06]	W=00000000	A=993e30c1	B=3fc7ca40	C=b3f52677	D=f3763846	E=284e43c2
[Round 07]	W=00000000	A=9e8c07d4	B=993e30c1	C=0ff1f290	D=b3f52677	E=f3763846
[Round 08]	W=00000000	A=4b6ae328	B=9e8c07d4	C=664f8c30	D=0ff1f290	E=b3f52677
[Round 09]	W=00000000	A=8351f929	B=4b6ae328	C=27a301f5	D=664f8c30	E=0ff1f290
[Round 10]	W=00000000	A=fbda9e89	B=8351f929	C=12dab8ca	D=27a301f5	E=664f8c30
[Round 11]	W=00000000	A=63188fe4	B=fbda9e89	C=60d47e4a	D=12dab8ca	E=27a301f5
[Round 12]	W=00000000	A=4607b664	B=63188fe4	C=7ef6a7a2	D=60d47e4a	E=12dab8ca
[Round 13]	W=00000000	A=9128f695	B=4607b664	C=18c623f9	D=7ef6a7a2	E=60d47e4a
[Round 14]	W=00000000	A=196bee77	B=9128f695	C=1181ed99	D=18c623f9	E=7ef6a7a2
[Round 15]	W=00000018	A=20bdd62f	B=196bee77	C=644a3da5	D=1181ed99	E=18c623f9
[Round 16]	W=c2c4c700	A=4e925823	B=20bdd62f	C=c65afb9d	D=644a3da5	E=1181ed99
[Round 17]	W=00000000	A=82aa6728	B=4e925823	C=c82f758b	D=c65afb9d	E=644a3da5
[Round 18]	W=00000030	A=dc64901d	B=82aa6728	C=d3a49608	D=c82f758b	E=c65afb9d
[Round 19]	W=85898e01	A=fd9e1d7d	B=dc64901d	C=20aa99ca	D=d3a49608	E=c82f758b
[Round 20]	W=00000000	A=1a37b0ca	B=fd9e1d7d	C=77192407	D=20aa99ca	E=d3a49608
[Round 21]	W=00000060	A=33a23bfc	B=1a37b0ca	C=7f67875f	D=77192407	E=20aa99ca
[Round 22]	W=0b131c03	A=21283486	B=33a23bfc	C=868dec32	D=7f67875f	E=77192407
[Round 23]	W=00000030	A=d541f12d	B=21283486	C=0ce88eff	D=868dec32	E=7f67875f
[Round 24]	W=85898ec1	A=c7567dc6	B=d541f12d	C=884a0d21	D=0ce88eff	E=868dec32
[Round 25]	W=16263806	A=48413ba4	B=c7567dc6	C=75507c4b	D=884a0d21	E=0ce88eff
[Round 26]	W=00000000	A=be35fbd5	B=48413ba4	C=b1d59f71	D=75507c4b	E=884a0d21
[Round 27]	W=00000180	A=4aa84d97	B=be35fbd5	C=12104ee9	D=b1d59f71	E=75507c4b
[Round 28]	W=2c4c700c	A=8370b52e	B=4aa84d97	C=6f8d7ef5	D=12104ee9	E=b1d59f71
[Round 29]	W=000000f0	A=c5fbaf5d	B=8370b52e	C=d2aa1365	D=6f8d7ef5	E=12104ee9
[Round 30]	W=93afb507	A=1267b407	B=c5fbaf5d	C=a0dc2d4b	D=d2aa1365	E=6f8d7ef5
[Round 31]	W=5898e048	A=3b845d33	B=1267b407	C=717eebd7	D=a0dc2d4b	E=d2aa1365
[Round 32]	W=8e9a9202	A=046faa0a	B=3b845d33	C=c499ed01	D=717eebd7	E=a0dc2d4b
[Round 33]	W=00000600	A=2c0ebc11	B=046faa0a	C=cee1174c	D=c499ed01	E=717eebd7
[Round 34]	W=b131c0f0	A=21796ad4	B=2c0ebc11	C=811bea82	D=cee1174c	E=c499ed01
[Round 35]	W=16263bc6	A=dcbbb0cb	B=21796ad4	C=4b03af04	D=811bea82	E=cee1174c
[Round 36]	W=4ebd41e	A=0f511fd8	B=dcbbb0cb	C=085e5ab5	D=4b03af04	E=811bea82
[Round 37]	W=626380a1	A=dc63973f	B=0f511fd8	C=f72eec32	D=085e5ab5	E=4b03af04
[Round 38]	W=16263806	A=4c986405	B=dc63973f	C=03d447f6	D=f72eec32	E=085e5ab5
[Round 39]	W=000018c0	A=32de1cba	B=4c986405	C=f718e5cf	D=03d447f6	E=f72eec32
[Round 40]	W=d2e138c4	A=fc87dedf	B=32de1cba	C=53261901	D=f718e5cf	E=03d447f6
[Round 41]	W=00000f00	A=970a0d5c	B=fc87dedf	C=8cb7872e	D=53261901	E=f718e5cf
[Round 42]	W=3afb5079	A=7f193dc5	B=970a0d5c	C=ff21f7b7	D=8cb7872e	E=53261901
[Round 43]	W=898e04e5	A=ee1b1aaf	B=7f193dc5	C=25c28357	D=ff21f7b7	E=8cb7872e
[Round 44]	W=e2ba3c2b	A=40f28e09	B=ee1b1aaf	C=5fc64f71	D=25c28357	E=ff21f7b7
[Round 45]	W=000060c0	A=1c51e1f2	B=40f28e09	C=fb86c6ab	D=5fc64f71	E=25c28357
[Round 46]	W=053a37cd	A=a01b846c	B=1c51e1f2	C=503ca382	D=fb86c6ab	E=5fc64f71

```
[Round 47] W=74458547 | A=bead02ca B=a01b846c C=8714787c D=503ca382 E=fb86c6ab
[Round 48] W=da9415ed | A=baef39337 B=bead02ca C=2806e11b D=8714787c E=503ca382
[Round 49] W=26380a16 | A=120731c5 B=baef39337 C=afab40b2 D=2806e11b E=8714787c
[Round 50] W=626383a1 | A=641db2ce B=120731c5 C=eebce4cd D=afab40b2 E=2806e11b
[Round 51] W=4ebf54de | A=3847ad66 B=641db2ce C=4481cc71 D=eebce4cd E=afab40b2
[Round 52] W=3835b44b | A=e490436d B=3847ad66 C=99076cb3 D=4481cc71 E=eebce4cd
[Round 53] W=0000f600 | A=27e9f1d8 B=e490436d C=8e11eb59 D=99076cb3 E=4481cc71
[Round 54] W=1e84c7a3 | A=7b71f76d B=27e9f1d8 C=792410db D=8e11eb59 E=99076cb3
[Round 55] W=98e04d98 | A=5e6456af B=7b71f76d C=09fa7c76 D=792410db E=8e11eb59
[Round 56] W=651d16a0 | A=c846093f B=5e6456af C=5edc7ddb D=09fa7c76 E=792410db
[Round 57] W=62658ca1 | A=d262ff50 B=c846093f C=d79915ab D=5edc7ddb E=09fa7c76
[Round 58] W=458544d6 | A=09d785fd B=d262ff50 C=f211824f D=d79915ab E=5edc7ddb
[Round 59] W=44584cb7 | A=3f52de5a B=09d785fd C=3498bfd4 D=f211824f E=d79915ab
[Round 60] W=7ba06619 | A=d756c147 B=3f52de5a C=4275e17f D=3498bfd4 E=f211824f
[Round 61] W=6380aea2 | A=548c9cb2 B=d756c147 C=8fd4b796 D=4275e17f E=3498bfd4
[Round 62] W=0ae55269 | A=b66c020b B=548c9cb2 C=f5d5b051 D=8fd4b796 E=4275e17f
[Round 63] W=627b49a1 | A=6b61c9e1 B=b66c020b C=9523272c D=f5d5b051 E=8fd4b796
[Round 64] W=7cd45c9d | A=19dfa7ac B=6b61c9e1 C=ed9b0082 D=9523272c E=f5d5b051
[Round 65] W=000f0000 | A=101655f9 B=19dfa7ac C=5ad87278 D=ed9b0082 E=9523272c
[Round 66] W=fb50753a | A=0c3df2b4 B=101655f9 C=0677e9eb D=5ad87278 E=ed9b0082
[Round 67] W=ec6765e8 | A=78dd4d2b B=0c3df2b4 C=4405957e D=0677e9eb E=5ad87278
[Round 68] W=ba3c2be2 | A=497093c0 B=78dd4d2b C=030f7cad D=4405957e E=0677e9eb
[Round 69] W=0060c000 | A=3f2588c2 B=497093c0 C=de37534a D=030f7cad E=4405957e
[Round 70] W=3a37cd05 | A=c199f8c7 B=3f2588c2 C=125c24f0 D=de37534a E=030f7cad
[Round 71] W=458546f4 | A=39859de7 B=c199f8c7 C=8fc96230 D=125c24f0 E=de37534a
[Round 72] W=b8599dd6 | A=edb42de4 B=39859de7 C=f0667e31 D=8fc96230 E=125c24f0
[Round 73] W=380a1a26 | A=11793f6f B=edb42de4 C=ce616779 D=f0667e31 E=8fc96230
[Round 74] W=01e02203 | A=5ee76897 B=11793f6f C=3b6d0b79 D=ce616779 E=f0667e31
[Round 75] W=e7cc3456 | A=63f7dab7 B=5ee76897 C=c45e4fdb D=3b6d0b79 E=ce616779
[Round 76] W=e6e60b69 | A=a079b7d9 B=63f7dab7 C=d7b9da25 D=c45e4fdb E=3b6d0b79
[Round 77] W=00f60a00 | A=860d21cc B=a079b7d9 C=d8fd6ad D=d7b9da25 E=c45e4fdb
[Round 78] W=5795ef4f | A=5738d5e1 B=860d21cc C=681e6df6 D=d8fd6ad E=d7b9da25
[Round 79] W=822e0879 | A=42541b35 B=5738d5e1 C=21834873 D=681e6df6 E=d8fd6ad
Block Result : A=a9993e36 B=4706816a C=ba3e2571 D=7850c26c E=9cd0d89d
-----
最终结果 (Hex): a9993e364706816aba3e25717850c26c9cd0d89d
```

2.2 结果验证

使用标准 SHA-1 工具验证结果：

输入：字符串"abc"

预期输出：a9993e36 4706816a ba3e2571 7850c26c 9cd0d89d

实验输出：a9993e36 4706816a ba3e2571 7850c26c 9cd0d89d

结论：完全一致，算法实现正确

三、实验总结与收获

3.1 学习收获

密码学算法理解

通过手动实现 SHA-1，深入理解了哈希函数的单向性、抗碰撞性和雪崩效应等核心特性。学习了如何将任意长度数据填充到固定块大小，包括"1"位添加、"0"位填充和长度追加的标准流程。通过 80 轮循环运算，理解了如何通过多轮迭代和逻辑函数组合实现高强度的散列效果。

编程技能提升

熟练使用位运算(<<, >>, &, |, ^, ~)。掌握循环左移宏的实现: $\text{ROTL}(x, n) = ((x) \ll (n)) | ((x) \gg (32 - (n)))$

进一步理解了大端序和小端序的区别。掌握了 `bytes_to_uint32` 和 `uint32_to_bytes` 转换函数。了解了网络字节序（大端序）在密码学中的重要性

3.2 实验难点与解决方案

1. 消息填充逻辑需要精确计算填充位置和长度表示

解决方案: 仔细研究 RFC 3174 标准, 使用 `context->count` 记录位数

2. 字节序处理在不同平台确保大端序一致性

解决方案: 使用宏函数统一转换, 避免依赖平台特性

解决本次 SHA-1 哈希算法实现实验不仅让我掌握了具体的算法实现技术, 更建立了系统化的密码学思维, 培养了严谨的编程习惯, 边界检查、错误处理、代码优化, 增强了解决复杂问题的能力, 激发了进一步学习兴趣。通过将理论知识与实践操作相结合, 我对哈希算法的工作原理、设计思想和实现细节有了深入理解, 为后续学习更高级的密码学知识打下了坚实基础。

实验二：公钥证书的颁发和使用

一、实验原理

在虚拟机安装 ssh 服务

```
sudo apt install openssh-server
```

安装客户端

```
udo apt install openssh-client
```

主机上生成密钥对

```
ssh-keygen -t rsa -b 4096 -f C:\Users\郑墨涵\.ssh\id_rsa
```

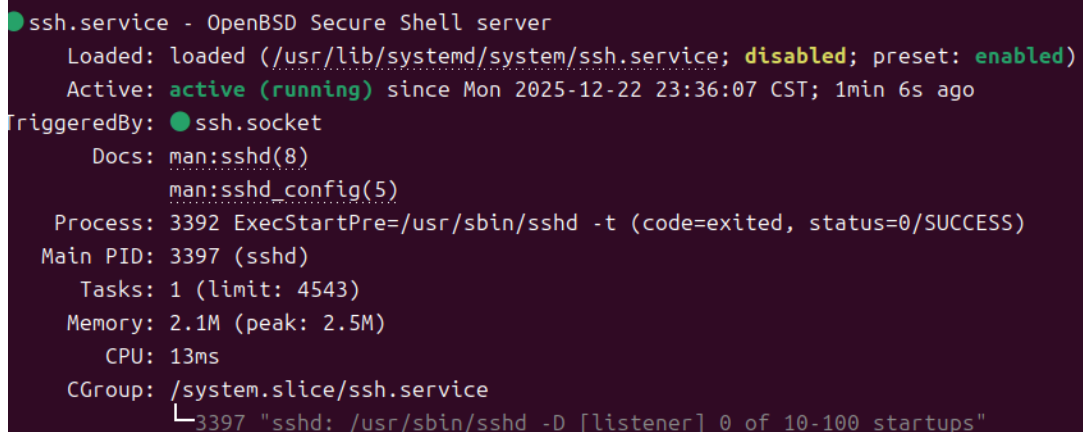


```
C:\Windows\System32>ssh-keygen -t rsa -b 4096 -f C:\Users\郑墨涵\.ssh\id_rsa
Generating public/private rsa key pair.
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in C:\Users\郑墨涵\.ssh\id_rsa
Your public key has been saved in C:\Users\郑墨涵\.ssh\id_rsa.pub
The key fingerprint is:
SHA256:hcoaQ1gI8y91BMaHSVpr0iZyS1MXvHgN+N/AT9BjIAM 郑墨涵@LAPTOP-L7V2IT9Q
The key's randomart image is:
+---[RSA 4096]-----+
|
|o. +EB= .o
|o. *=o+.,.+
|+ *+o=.o..
|B.++.+.
|o 0+.o.S=
|.= + .o
|..
+---[SHA256]-----+
```

在虚拟机上配置 ssh

```
service ssh start
```

```
service ssh status
```



```
● ssh.service - OpenBSD Secure Shell server
   Loaded: loaded (/usr/lib/systemd/system/ssh.service; disabled; preset: enabled)
   Active: active (running) since Mon 2025-12-22 23:36:07 CST; 1min 6s ago
     TriggeredBy: ● ssh.socket
       Docs: man:sshd(8)
             man:sshd_config(5)
    Process: 3392 ExecStartPre=/usr/sbin/sshd -t (code=exited, status=0/SUCCESS)
   Main PID: 3397 (sshd)
      Tasks: 1 (limit: 4543)
     Memory: 2.1M (peak: 2.5M)
        CPU: 13ms
    CGroup: /system.slice/ssh.service
            └─3397 "sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups"
```

查看虚拟机 ip

```
ip addr
```



```
d0belr@d0belr-VMware-Virtual-Platform:~$ ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:86:ff:5b brd ff:ff:ff:ff:ff:ff
    altname ens33
    inet 192.168.242.128/24 brd 192.168.242.255 scope global dynamic noprefixroute ens33
        valid_lft forever preferred_lft 1592sec
    inet6 fe80::20c:29ff:fe86:ff5b/64 scope link
        valid_lft forever preferred_lft forever
```

Ping 虚拟机

ping 192.168.242.128

```
C:\Windows\System32>ping 192.168.242.128

正在 Ping 192.168.242.128 具有 32 字节的数据:
来自 192.168.242.128 的回复: 字节=32 时间<1ms TTL=64
来自 192.168.242.128 的回复: 字节=32 时间=1ms TTL=64
来自 192.168.242.128 的回复: 字节=32 时间=1ms TTL=64
来自 192.168.242.128 的回复: 字节=32 时间=1ms TTL=64

192.168.242.128 的 Ping 统计信息:
    数据包: 已发送 = 4, 已接收 = 4, 丢失 = 0 (0% 丢失),
    最短 = 0ms, 最长 = 1ms, 平均 = 0ms
```

复制公钥

在 Windows 上打开 id_rsa.pub ， 复制文件内容。

```
C:\Users\郑晓精> ssh > ❷ id_rsa.pub
1  ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAQCM1SnuoVirPz8me/AQ4rYrziQqhAkApYHxak7J+lVol1Da1LaJ5pCkobJc/na4VQQAGnnoggAvRQHuDABgsbdEM0Cx1bU
2
```

在虚拟机中运行以下命令：

mkdir -p ~/.ssh

chmod 700 ~/.ssh

nano ~/.ssh/authorized_keys

保存退出

```
d0belr@d0belr-VMware-Virtual-Platform:~$ mkdir -p ~/.ssh
d0belr@d0belr-VMware-Virtual-Platform:~$ chmod 700 ~/.ssh
d0belr@d0belr-VMware-Virtual-Platform:~$ nano ~/.ssh/authorized_keys
d0belr@d0belr-VMware-Virtual-Platform:~$ nano ~/.ssh/authorized_keys
d0belr@d0belr-VMware-Virtual-Platform:~$ chmod 600 ~/.ssh/authorized_keys
d0belr@d0belr-VMware-Virtual-Platform:~$
```

设置文件权限

chmod 600 ~/.ssh/authorized_keys

二、实验结果

免密登录

```
ssh -i C:\Users\郑墨涵\.ssh\id_rsa d0belr@192.168.242.128
```

```
C:\Windows\System32>ssh -i C:\Users\郑墨涵\.ssh\id_rsa d0belr@192.168.242.128
Welcome to Ubuntu 24.04.3 LTS (GNU/Linux 6.14.0-34-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

扩展安全维护 (ESM) Applications 未启用。

0 更新可以立即应用。

启用 ESM Apps 来获取未来的额外安全更新
请参见 https://ubuntu.com/esm 或者运行: sudo pro status
```

三、实验总结与收获

通过 SSH 服务密钥登录虚拟机的实践，我获得了对非对称加密技术在实际应用中的深刻理解，这不仅是一次技术操作，更是对现代网络安全体系的全面认知。从最初生成 RSA 密钥对到成功实现免密码登录，整个过程让我直观体会到公钥密码学的精妙设计：私钥作为绝对保密的核心，必须妥善保管在本地；而公钥则可以放心分发到目标服务器，这种不对称性正是安全通信的基石。通过亲手配置 `authorized_keys` 文件，我明白了 SSH 协议如何利用这一机制实现身份验证——客户端用私钥签名，服务器用存储的公钥验证，完全无需传输任何秘密信息。这一过程让我认识到安全协议设计的严谨性，特别是对文件权限的严格要求（如 `~/.ssh` 目录必须为 700 权限，私钥必须为 600），这些看似琐碎的细节实际上是防止密钥泄露的重要防线。

在实验调试过程中，我遇到了连接失败、权限错误等问题，通过查看系统日志、使用 `ssh -v` 调试模式，逐渐掌握了故障排查的方法论。这让我体会到，安全不仅仅是理论设计，更是工程实践的积累，每一个配置项都有其安全考量，比如禁用密码登录可以彻底杜绝暴力破解风险，更换默认端口能减少自动化攻击扫描。更深入的是，我理解了 SSH 服务的多层面安全架构：传输层加密保护通信内容，主机密钥验证防止中间人攻击，用户密钥实现身份认证，这种分层设计体现了纵深防御的安全理念。这种从理论到实践的跨越，不仅加深了我对密码学原理的理解，更培养了我将安全理论转化为具体实施的能力，为今后设计、部署和维护安全系统奠定了坚实基础。

实验三：TLS 协议分析

一、实验原理

Ping cn.bing.com

```
C:\Users\郑墨涵>ping cn.bing.com

正在 Ping china.bing123.com [202.89.233.100] 具有 32 字节的数据:
来自 202.89.233.100 的回复: 字节=32 时间=22ms TTL=116
来自 202.89.233.100 的回复: 字节=32 时间=27ms TTL=116
来自 202.89.233.100 的回复: 字节=32 时间=23ms TTL=116
来自 202.89.233.100 的回复: 字节=32 时间=31ms TTL=116

202.89.233.100 的 Ping 统计信息:
    数据包: 已发送 = 4, 已接收 = 4, 丢失 = 0 (0% 丢失),
    往返行程的估计时间(以毫秒为单位):
        最短 = 22ms, 最长 = 31ms, 平均 = 25ms
```

Edge 访问 cn.bing.com,wireshark 抓包

筛选 ip.addr == 202.89.233.100 and tls

TLSv1.2	692 Client Hello (SNI=r.bing.com)
TCP	1444 443 → 58188 [ACK] Seq=1 Ack=2029 Win=4194816 Len=1390 [TCP PDU reassembled in 173]
TLSv1.2	333 Server Hello, Certificate, Certificate Status, Server Key Exchange, Server Hello Done
TLSv1.2	212 Client Key Exchange, Change Cipher Spec, Encrypted Handshake Message
TLSv1.2	396 New Session Ticket, Change Cipher Spec, Encrypted Handshake Message
TLSv1.2	123 Application Data

Client Hello: 客户端向服务端打招呼；携带支持的协议、支持的安全套件供服务端选择；**Server Hello:** 服务端回应客户客户端的招呼信息；结合客户端的信息，选择合适的加密套件；

Certificate: 服务端向客户端发送自己的数字证书（此证书包含服务端的公钥），以实现验证身份；

Server Key Exchange: 服务端向客户端发送基于选择的加密套件生成的公钥（此公钥为椭圆曲线的公 钥，用于协商出对称加密的密钥）；

Server Hello Done: 服务端向客户端表示响应结束；

Client Key Exchange: 客户端向服务端发送自己生成的公钥（此公钥为椭圆曲线的公钥，用于协商出 对称加密的密钥）；

Change Cipher Spec: 变更密码规范；告知服务端/客户端，以后的通信都是基于 AES 加密的；

Encrypted Handshake Message: 基于协商生成的密钥，用 AES 加密验证信息让服务端/客户端进行认证；如果对方可以解密，则双方认证无误开始通信；

二、实验结果

```
Transport Layer Security
[Stream index: 3]
  TLSv1.2 Record Layer: Handshake Protocol: Client Hello
    Content Type: Handshake (22)
    Version: TLS 1.0 (0x0301)
    Length: 2023
    Handshake Protocol: Client Hello
      Handshake Type: Client Hello (1)
      Length: 2019
      Version: TLS 1.2 (0x0303)
      Random: 318d34e1f72ce86e6141ee1b303ce9b347c76dbf958d1c87acbec7ee7b96be9e
      Session ID Length: 32
      Session ID: 1181547ab8c8c8489695a640da5b422b9bee1aa54be8af9b6bf8b503ba27
      Cipher Suites Length: 32
      Cipher Suites (16 suites)
```

```
Transport Layer Security
[Stream index: 3]
  TLSv1.2 Record Layer: Handshake Protocol: Multiple Handshake Messages
    Content Type: Handshake (22)
    Version: TLS 1.2 (0x0303)
    Length: 7224
    Handshake Protocol: Server Hello
      Handshake Type: Server Hello (2)
      Length: 98
      Version: TLS 1.2 (0x0303)
      Random: 69497a5ec00b5098cb24e9ac771612ad580283562755001e47809d22a7caa0da
      Session ID Length: 32
      Session ID: c01d0000bec0af83b1b84cc9bf018db966f6f8993ffde96a39a7e42e1f71be62
      Cipher Suite: TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384 (0xc030)
      Compression Method: null (0)
```

三、实验总结与收获

通过利用抓包软件进行 TLS 协议分析实验，我对现代网络安全通信的底层机制有了前所未有的直观认知和深刻理解。通过分析密钥交换报文，我亲眼看到前向保密（PFS）如何在实际中实现：临时密钥对的生成和交换确保即使长期私钥未来泄露，已记录的加密会话也不会被破解。在 ChangeCipherSpec 那个微妙而关键的转折点上，我感受到了协议状态机的精确切换——从此之后的所有通信都进入了对称加密的快车道，而之前的所有协商都是为了安全地建立那组会话密钥。Finished 报文中的加密验证则确保整个握手过程没有被篡改，这种端到端的完整性验证让我理解了双向认证的完备性。

在实验中，我还意外捕捉到了一些协议扩展的细节，比如服务器名称指示

（SNI）如何在明文部分暴露访问的域名，这让我同时意识到了 TLS 协议在安全与功能之间的权衡。通过尝试解密 TLS 流量（在拥有密钥的情况下），我进一步理解了记录层协议的分帧机制和不同密码套件的实际加密效果。这种从数据包级别理解安全协议的经历，让我对网络安全工程师的工作产生了更深的敬意——他们必须像协议设计者一样思考，像密码学家一样严谨，像侦探一样敏锐，才能在复杂的网络环境中筑起真正的安全防线。

实验四 WEB 用户名口令解析

一、实验原理

HTTP 协议是一种明文传输协议，不提供任何方式的数据加密。

当用户通过表单提交登录信息时，浏览器通常使用 POST 方法将数据发送到服务器。

这些数据以明文形式出现在 HTTP 请求体中，格式通常为键值对（如 username=admin&password=123456）。

使用抓包软件（如 Wireshark）可以捕获到这些数据包，通过分析数据包的内容，我们可以提取出用户名和口令。

Ping wlsy.xidian.edu.cn

```
C:\Users\郑墨涵>ping wlsy.xidian.edu.cn

正在 Ping wlsy.xidian.edu.cn [202.117.120.6] 具有 32 字节的数据:
来自 202.117.120.6 的回复: 字节=32 时间=3ms TTL=57
来自 202.117.120.6 的回复: 字节=32 时间=3ms TTL=57
来自 202.117.120.6 的回复: 字节=32 时间=3ms TTL=57
来自 202.117.120.6 的回复: 字节=32 时间=2ms TTL=57

202.117.120.6 的 Ping 统计信息:
    数据包: 已发送 = 4, 已接收 = 4, 丢失 = 0 (0% 丢失),
    往返行程的估计时间(以毫秒为单位):
        最短 = 2ms, 最长 = 3ms, 平均 = 2ms
```

筛选 http and ip.addr == 202.117.120.6

开始抓包

输入账号和密码登录



停止抓包并筛选信息

三、实验结果

可见

"login1\$StuLoginID"="23009201247"

"login1\$StuPassword"="wjtz2040"

说明解析成功

```
▶ Frame 983: Packet, 1416 bytes on wire (11328 bits), 1416 bytes captured (11328 bits) on interface \Device\NPF_{7AD63F5D-353A-40...
▶ Ethernet II, Src: Intel_78:84:b7 (04:e8:b9:78:84:b7), Dst: HuaweiTechno_21:30:ab (b0:76:1b:21:30:ab)
▶ Internet Protocol Version 4, Src: 10.198.132.124, Dst: 202.117.120.6
▶ Transmission Control Protocol, Src Port: 53905, Dst Port: 80, Seq: 1843, Ack: 16039, Len: 1362
▼ Hypertext Transfer Protocol
  POST /phyews/default.aspx?ReturnUrl=%2fPhyEws%2fstudent%2fstudent.aspx HTTP/1.1\r\n
    Request Method: POST
    Request URI: /phyews/default.aspx?ReturnUrl=%2fPhyEws%2fstudent%2fstudent.aspx
    Request URI Path: /phyews/default.aspx
    Request URI Query: ReturnUrl=%2fPhyEws%2fstudent%2fstudent.aspx
    Request URI Query Parameter: ReturnUrl=%2fPhyEws%2fstudent%2fstudent.aspx
    Request Version: HTTP/1.1
    Host: wlsy.xidian.edu.cn\r\n
    Connection: keep-alive\r\n
    Content-Length: 585\r\n
    Cache-Control: max-age=0\r\n
    Upgrade-Insecure-Requests: 1\r\n
    Origin: http://wlsy.xidian.edu.cn\r\n
    Content-Type: application/x-www-form-urlencoded\r\n
    User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/127.0.0.0 Safari/537.36
    Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.9\r\n
    Referer: http://wlsy.xidian.edu.cn/phyews/default.aspx?ReturnUrl=%2fPhyEws%2fstudent%2fstudent.aspx\r\n
    Accept-Encoding: gzip, deflate\r\n
    Accept-Language: zh-CN,zh;q=0.9,en;q=0.8,en-GB;q=0.7,en-US;q=0.6\r\n
    \r\n
    [Response in frame: 997]
    [Full request URI: http://wlsy.xidian.edu.cn/phyews/default.aspx?ReturnUrl=%2fPhyEws%2fstudent%2fstudent.aspx]
    File Data: 585 bytes
  ▼ HTML Form URL Encoded: application/x-www-form-urlencoded
    ▶ Form item: "__EVENTTARGET" = ""
    ▶ Form item: "__EVENTARGUMENT" = ""
    ▶ Form item: "__VIEWSTATE" = "/wEPDwUKMTEzNzM0MjM0OWQYAUeX19Db250cm9sc1JlcXVpcmVQb3N0QmFja0tleV9fFgEFD2xvZ21uMSRidG5Mb2dpbkOu...
    ▶ Form item: "__VIEWSTATEGENERATOR" = "EE008CD9"
    ▶ Form item: "EVENTVALIDATION" = "/wEPDwUKMTEzNzM0MjM0OWQYAUeX19Db250cm9sc1JlcXVpcmVQb3N0QmFja0tleV9fFgEFD2xvZ21uMSRidG5Mb2dpbkOu...
    ▶ Form item: "login1$StuLoginID" = "23009201247"
    ▶ Form item: "login1$StuPassword" = "wjtz2040"
    ▶ Form item: "login1$userRole" = "Student"
    ▶ Form item: "login1$btnLogin.x" = "23"
    ▶ Form item: "login1$btnLogin.y" = "11"
```

四、实验总结与收获

通过使用抓包软件实现对 WEB 用户名口令的解析实验,我获得了对网络通信安全性的深刻认识和实际网络安全风险的直观体验。当我在 WireShark 中成功捕获到 HTTP 明文登录请求并解析出其中的用户名和密码时,我明白了原来在未加密的 HTTP 协议下,用户的敏感信息就像明信片一样在网络中毫无遮掩地传递,任何能够访问网络数据流的人都能轻易读取这些隐私内容。这一发现让我彻底理解了为什么现代互联网如此强调 HTTPS 的普及,因为 HTTP 协议的根本缺陷在于它缺乏端到端的加密保护,使得用户的登录凭证、个人信息乃至金融数据在网络

传输的每一个中间节点上都处于暴露状态。

通过对比 HTTP 与 HTTPS 的数据包，我直观地看到了加密通信的巨大差异：在 HTTPS 流量中，除了握手阶段的证书交换等元数据外，应用层数据完全被加密保护，呈现为看似随机的二进制流，这种“不可读性”恰恰是安全性的体现。这一对比让我更加理解了 TLS/SSL 协议在现代网络安全中的基石作用，也让我认识到为什么各个主流浏览器都在大力推动全站 HTTPS，甚至将 HTTP 标记为“不安全”。实验中的每一个被成功解析的密码都像一个警钟，提醒着我作为未来可能参与系统开发或安全运维的人员，必须将数据加密和隐私保护作为设计的首要原则。

这次实验培养了我对用户数据安全的责任感——每一个比特的敏感信息都值得用最强的加密技术来保护，每一行代码都可能影响成千上万用户的隐私安全。这种认识将伴随我未来的整个职业生涯，让我始终将安全性放在技术决策的核心位置。